



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Reinforcement Learning Based Control of Batch Reactor using PPO

Unsupervised Intelligence in Cyber Physical System

Group Number – 3

Sinchana Prakash Naik (12) - 230961018

Chinmayi(13) - 230961022

Rashi Singh Rawat (14)- 230961024

Anushka Salgaonkar(16) - 230961028

Aim :

To implement a reinforcement learning-based control strategy using Proximal Policy Optimization (PPO) for a batch reactor and achieve accurate set-point tracking with stable and efficient learning.

Objective :

- To understand the fundamentals of reinforcement learning and the PPO algorithm
- To model the batch reactor as a reinforcement learning environment
- To design a reward function that minimizes set-point tracking error
- To implement PPO with a clipped surrogate objective for stable policy updates
- To train an agent to learn optimal control actions
- To evaluate system performance based on tracking accuracy and stability

Methodology:

Problem Definition

The batch reactor is modeled as a dynamic control system where the output must track a desired set-point. Maintaining this condition is important for process efficiency and product quality. A reinforcement learning-based controller is developed to automatically adjust control inputs and minimize tracking errors.

Reinforcement Learning Framework

The problem is formulated as a reinforcement learning task consisting of state, action, and reward:

- State (st): Current reactor output
- Action (at): Control input applied to the reactor
- Reward (rt): Negative squared tracking error

$$r_t = -(y_t - y_{setpoint})^2$$

This reward formulation ensures that the agent learns to minimize tracking error.

Environment Modeling

The reactor dynamics are approximated using a first-order system:

$$x_{t+1} = x_t + 0.1(-x_t + u_t)$$

This equation represents how the reactor output evolves over time based on control input.

PPO Algorithm Implementation

The PPO algorithm is used to learn the optimal control policy. It follows an actor-critic approach, where:

- The policy network (actor) selects actions
- The value function (critic) estimates future rewards

PPO improves training stability using a clipped surrogate objective function that restricts large policy updates.

Training Process

The agent interacts with the environment over multiple episodes:

1. Collect trajectories by interacting with the environment
2. Compute rewards and estimate advantages
3. Update the policy using minibatch stochastic gradient ascent
4. Repeat updates over multiple epochs for better learning

Additional loss components such as value loss and entropy bonus may be included to improve stability and exploration.

Performance Evaluation

The trained model is evaluated using:

- Set-point tracking performance
- Response speed
- Stability and smoothness of control

A successful controller minimizes steady-state error and achieves stable tracking.

ALGORITHM:

Proximal Policy Optimization (PPO) for Batch Reactor Control

Initialization

- The Proximal Policy Optimization (PPO) algorithm begins by initializing the parameters of two neural networks: the policy network (actor), represented by θ , and the value function network (critic), represented by ϕ .
- The actor network is responsible for selecting control actions based on the current state of the reactor, while the critic network evaluates the expected future rewards of those states. These networks are typically initialized with random weights and are updated iteratively during training.

Environment Setup

- The batch reactor is modeled as a dynamic system where the state evolves over time based on the applied control input. The system dynamics are approximated using a first-order equation:

$$x_{t+1} = x_t + 0.1(-x_t + u_t)$$

where x_t represents the current reactor output and u_t represents the control input.

- The reward function is designed to minimize the deviation between the reactor output and the desired setpoint:

$$r_t = (y_t - y_{setpoint})^2$$

- This ensures that the agent is penalized for large tracking errors and encouraged to maintain the reactor output close to the setpoint.

Interaction (Data Collection)

At each timestep t , the agent interacts with the environment to collect training data. The following steps are performed:

- The current state s_t is observed
- An action a_t is sampled from the policy distribution $\pi_{\theta}(a_t | s_t)$
- The action is applied to the reactor system
- The environment transitions to a new state s_{t+1}
- A reward r_t is obtained based on tracking performance
- The transition (s_t, a_t, r_t, s_{t+1}) is stored for learning.
- This process is repeated over multiple timesteps to generate trajectories of experience.

Compute Temporal Difference Error

- The Temporal Difference (TD) error is calculated to estimate the immediate improvement of the value function:

$$\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

- where γ is the discount factor. This term represents the difference between the predicted value and the observed reward plus future estimate.

Compute Advantage (Generalized Advantage Estimation)

- To improve learning stability and reduce variance, the advantage function is computed using Generalized Advantage Estimation (GAE):

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

- where λ is a smoothing parameter. The advantage function indicates how beneficial a particular action is compared to the average expected outcome.

Compute Probability Ratio

- The PPO algorithm measures the change in policy using the probability ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

- This ratio compares the likelihood of selecting an action under the new policy versus the old policy.

PPO Clipped Objective

- The core idea of PPO is to prevent large policy updates that may destabilize learning. This is achieved using the clipped objective function:

$$L_{\text{clip}}(\theta) = E[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

- The clipping function restricts the probability ratio within a fixed range, ensuring that updates remain small and stable.

Value Function Loss

- The critic network is trained to accurately estimate the value of each state. This is done by minimizing the value function loss:

$$L_{VF} = (V\phi(s_t) - V_{\text{target}})^2$$

- where V_{target} represents the observed return.

Entropy Bonus (for Exploration)

- To encourage exploration and prevent premature convergence, an entropy term is added:

$$S(\pi\theta)$$

- This term promotes randomness in action selection, allowing the agent to explore different control strategies during training.

Total Objective Function

- The final objective function combines the policy loss, value function loss, and entropy bonus

$$L(\theta, \phi) = E[L_{clip}(\theta) - c^1 L_{VF} + c^2 S(\pi\theta)]$$

- Where c_1 and c_2 are weighting coefficients that balance the different components of the loss.

Optimization Step

- The parameters θ and ϕ are updated using gradient-based optimization methods such as Adam. The updates are performed over multiple epochs using minibatches of collected data, which improves learning efficiency and stability.

Policy Update

- After each optimization step, the updated policy parameters replace the old policy:

$$\theta_{old} \leftarrow \theta$$

- This ensures that the next iteration uses the latest policy for data collection.

Repeat

- The entire process of data collection, advantage computation, and optimization is repeated for multiple iterations until the policy converges and achieves stable performance.

Output

- The final output of the PPO algorithm is an optimized policy π_{θ} that maps reactor states to control actions. This learned policy minimizes tracking error and ensures stable and efficient control of the batch reactor.

Simulation and Implementation

The PPO-based controller is implemented using a reinforcement learning framework. The batch reactor environment is modeled programmatically, where the agent interacts with the system by observing states and applying control actions.

The policy (actor) and value (critic) networks are implemented using a neural network architecture, typically consisting of fully connected layers. The agent is trained over multiple episodes, where each episode represents a sequence of interactions with the reactor system.

During training, the agent collects state-action-reward trajectories and updates its policy using the PPO algorithm. The optimization is performed using the Adam optimizer, and training is carried out over multiple epochs with minibatch updates to improve stability and convergence.

Hyperparameters such as learning rate, discount factor (γ), clipping parameter (ϵ), and GAE parameter (λ) are carefully chosen to ensure efficient learning.

